
Boris Cherny's CLAUDE.md File

The exact config the founder of Claude Code uses to turn Claude into an autonomous staff engineer.

What is CLAUDE.md?

CLAUDE.md is a markdown file you drop in your repo root. It tells Claude Code **how to think, plan, and execute** -- not just what to build.

Think of it as an operating system for your AI coding assistant.

Workflow Orchestration

1. Plan Mode Default

- Enter plan mode for ANY non-trivial task (3+ steps or architectural decisions)
- If something goes sideways, STOP and re-plan immediately
- Use plan mode for verification steps, not just building
- Write detailed specs upfront to reduce ambiguity

2. Subagent Strategy

- Use subagents liberally to keep main context window clean
- Offload research, exploration, and parallel analysis to subagents
- For complex problems, throw more compute at it via subagents
- One task per subagent for focused execution

3. Self-Improvement Loop

- After ANY correction from the user: update tasks/lessons.md with the pattern
- Write rules for yourself that prevent the same mistake
- Ruthlessly iterate on these lessons until mistake rate drops
- Review lessons at session start for relevant project

4. Verification Before Done

- Never mark a task complete without proving it works
- Diff behavior between main and your changes when relevant
- Ask yourself: "Would a staff engineer approve this?"
- Run tests, check logs, demonstrate correctness

5. Demand Elegance (Balanced)

- For non-trivial changes: pause and ask "is there a more elegant way?"
- If a fix feels hacky: "Knowing everything I know now, implement the elegant solution"
- Skip this for simple, obvious fixes -- don't over-engineer
- Challenge your own work before presenting it

6. Autonomous Bug Fixing

- When given a bug report: just fix it. Don't ask for hand-holding
- Point at logs, errors, failing tests -- then resolve them
- Zero context switching required from the user
- Go fix failing CI tests without being told how

Task Management

1. Plan First: Write plan to tasks/todo.md with checkable items
2. Verify Plan: Check in before starting implementation
3. Track Progress: Mark items complete as you go
4. Explain Changes: High-level summary at each step
5. Document Results: Add review section to tasks/todo.md
6. Capture Lessons: Update tasks/lessons.md after corrections

Core Principles

- Simplicity First: Make every change as simple as possible. Impact minimal code.

- No Laziness: Find root causes. No temporary fixes. Senior developer standards.
- Minimal Impact: Only touch what's necessary. No side effects with new bugs.

What's Inside

Section 1: Workflow Orchestration

6 rules that control how Claude Code approaches every task.

- **Plan Mode Default** -- Enter plan mode for any non-trivial task (3+ steps or architectural decisions)
- **Subagent Strategy** -- Use subagents liberally to keep the main context window clean
- **Self-Improvement Loop** -- After any correction, update lessons.md with the pattern
- **Verification Before Done** -- Never mark a task complete without proving it works
- **Demand Elegance** -- For non-trivial changes, pause and ask "is there a more elegant way?"
- **Autonomous Bug Fixing** -- When given a bug report, just fix it. No hand-holding.

Section 2: Task Management

6 steps Claude Code follows for every task.

1. **Plan First** -- Write plan to tasks/todo.md with checkable items
2. **Verify Plan** -- Check in before starting implementation
3. **Track Progress** -- Mark items complete as you go
4. **Explain Changes** -- High-level summary at each step
5. **Document Results** -- Add review section to tasks/todo.md
6. **Capture Lessons** -- Update tasks/lessons.md after corrections

Section 3: Core Principles

3 non-negotiable rules for every line of code.

- **Simplicity First** -- Make every change as simple as possible. Impact minimal code.
- **No Laziness** -- Find root causes. No temporary fixes. Senior developer standards.
- **Minimal Impact** -- Only touch what's necessary. No side effects with new bugs.

How to Use It

1. Create a file called `CLAUDE.md` in your project root
2. Paste these rules inside
3. Open Claude Code in your terminal
4. Watch it plan, execute, and verify like a senior engineer

Who is Boris Cherny?

Boris Cherny is the founder of Claude Code at Anthropic. This is the exact file he uses in his own repos to get the most out of Claude Code.

One Rule

Never skip Context or Stop Conditions.

Those two are the difference between a generic Claude answer and one that actually does the work.

Free n8n AI Agent Courses

1. [n8n Beginner Course \(Youtube\)](#)
2. [n8n Advanced Course \(Youtube\)](#)
3. [n8n Master Course \(Udemy\)](#)

You can find me here: **LinkedIn**, **Instagram**, **Youtube** and **Udemy**.

1. Master n8n AI Agents: Build & Sell AI Agents and Automations: [Click here](#)
2. AI Voice Agent Course using Retell AI, n8n, Make, Go Highlevel and Vapi AI: [Click here](#)
3. Cold Email MasterCourse: [Click here](#)
4. Rest of the Courses: [Click here](#)

Hope you found emm useful...Again it's just scratching the ground, this thing is rabbit hole

